

TRPO 实验报告

LAMDA-5

June 16, 2026

Abstract

本报告实现并分析了 Trust Region Policy Optimization (TRPO) 算法 [1]。我们完成了 `compute_advantages_from_traj`、`compute_advantages_from_rollout` 和 `_update_policy` 三部分代码补全，并在 Humanoid-v5、HalfCheetah-v5 和 Walker2d-v5 三个 MuJoCo 连续控制环境中进行了系统性实验。实验对比了 rollout 与 trajectory 两种采样方式，以及 advantage normalization 对训练稳定性的影响。结果表明：rollout 在样本利用率 (100%) 上优于 trajectory (62%–71%)，在 HalfCheetah 上最终回报明显更高；trajectory 在 Humanoid 等高维任务中能通过保留完整 episode 边界获得更高质量的 advantage 估计。Advantage normalization 对数值稳定性至关重要——在 Walker2d 中，关闭归一化后 surrogate objective 出现量级达 10^{33} 的非有限值，导致策略学习几乎失败。

1 实验概况

1.1 实验环境

本实验选取 **Humanoid-v5**、**HalfCheetah-v5** 和 **Walker2d-v5** 三个 MuJoCo Gymnasium v5 环境，对比以下四种配置：

- 采样方式：rollout 与 trajectory
- 优势归一化：`advan_norm=true` 与 `advan_norm=false`

1.2 超参数设置

主要超参数见表 1。

Table 1: 实验超参数设置

超参数	值
并行环境数	8
每 epoch 交互步数 (rollout)	2000 (共 $8 \times 2000 = 16000$ 个 transition)
轨迹条数 (trajectory)	10
最大 episode 长度	1000
总训练 epoch 数	5000
折扣因子 γ	0.99
GAE 参数 λ	0.95
KL 约束阈值 δ	0.01
共轭梯度迭代次数	10
Critic 更新轮数	5
策略/价值网络结构	2 层 MLP (64, 64), Tanh 激活

1.3 实验结果汇总

表 2 汇总了三个环境在各配置下的关键指标。”最终测试回报”取训练末期测试数据的均值；trajectory 模式因总交互步数与 rollout 不同，所报数值为相近交互步数处的测试结果，以 [†] 标注。

Table 2: 各配置实验结果汇总

环境	采样模式	Advan Norm	最终测试回报	样本利用率	平均有效步数/epoch
Humanoid-v5	rollout	✓	1083	100%	16000
	rollout		1085	100%	16000
	trajectory	✓	$\sim 655^{\dagger}$	69.0%	~ 885
HalfCheetah-v5	rollout	✓	4276	100%	16000
	rollout		4608	100%	16000
	trajectory	✓	$\sim 1727^{\dagger}$	62.5%	~ 10000
Walker2d-v5	rollout	✓	4718	100%	16000
	rollout		355	100%	16000
	trajectory	✓	$\sim 4713^{\dagger}$	70.8%	~ 6225

2 代码补全

2.1 collect_trajectories 实现

2.1.1 采样停止条件

问：collect_trajectories 与普通 rollout 采样方式相比，数据收集的停止条件分别是什么？

答：普通 rollout (interact_with_envs) 按固定步数截断：每个 epoch 每个并行环境交互满 interact_per_epoch 步即返回，不等 episode 结束。

collect_trajectories 按轨迹条数截断：外层循环条件为 while not self.traj_rollout.full 直到存满 trajnum 条轨迹才停止；每条轨迹以“环境终止”或“达到 max_episode_length”为内部结束条件。

2.1.2 轨迹数据与 Episode 边界处理

问：在 trajectory 采样模式下，每一条轨迹需要保存哪些信息？这些信息分别在后续 TRPO 更新中起什么作用？当环境返回终止信号时，轨迹又会如何结束？未达到最大长度的部分又应如何处理？

答：每条轨迹保存 states、actions、rewards、dones、log_probs，对应 add_traj() 的输入字段：

- states：供 actor 重算 $\log \pi_{\theta}(a|s)$ ，供 critic 估计 $V(s_t)$ ；
- actions：与 states 配合计算重要性采样比率；
- rewards：从后往前递推 GAE 和 return；
- dones：递推时乘以 $(1 - d_t)$ 切断 episode 边界的 bootstrap；
- log_probs：旧策略概率，用于计算 $\exp(\log \pi_{\theta} - \log \pi_{\theta_{\text{old}}})$ 。

环境返回终止信号时立即调用 add_traj() 保存当前轨迹并重置该 env 的缓冲区。未填满 max_episode_length 的剩余位置保持零初始化，masks 置为 0，后续用 masks==1 过滤掉 padding。

2.1.3 masks 的作用

问：TrajectoryRollout 中的 masks 变量有什么作用？如果不使用 masks，计算 return 或 advantage 时可能出现什么问题？

答: `masks[i,t]=1` 表示该位置是真实 transition, `=0` 表示 padding。`TrajectoryRollout` 预分配形状为 `(trajnum, max_episode_length)`, 短轨迹后必有零填充, `add_traj()` 中 `self.masks[pos,:traj_len]=1` 记录真实长度。

若不使用 `masks`, 会产生如下问题:

- padding 的零奖励和零状态污染 `return/advantage` 的均值与方差, `advantage normalization` 随之偏移;
- GAE 反向递推可能将无效步当成真实 transition, 导致 critic 和 actor 梯度失真;
- 样本利用率统计虚高, 掩盖实际有效 transition 数量。

2.1.4 采样后重置环境的必要性

问: `collect_trajectories` 在收集完一批轨迹后为什么需要重新初始化环境状态?

答: 当缓冲区满时, 某些环境可能停在 episode 中途, 其 `observations` 是非起点状态。若不重置, 下一轮轨迹从这些中间状态开始, 破坏“从初始分布出发”的假设, 轨迹边界不干净。`initialize()` 确保每轮采样从 episode 起点开始, 使 on-policy 的“采样—更新—清空”流程清晰, 避免上一批残留状态污染下一批新策略的数据。

2.2 _update_policy 实现

TRPO 的核心理论依赖如下定理:

Theorem 1 (Schulman et al. 2015)

令 $\alpha = D_{TV}^{\max}(\pi_{\text{old}}, \pi_{\text{new}})$, $\epsilon = \max_{s,a} |A_{\pi_{\text{old}}}(s,a)|$, 则下界成立:

$$\eta(\pi_{\text{new}}) \geq L_{\pi_{\text{old}}}(\pi_{\text{new}}) - \frac{4\epsilon\gamma}{(1-\gamma)^2} \alpha^2.$$

2.2.1 定理含义与单调改进条件

问: 解释该公式中每一项的实际意义, 这个公式在什么情况下可以保证策略的单调改进? 由此直观解释 TRPO 算法的设计思路。

答: 各项含义如下:

- $\eta(\pi_{\text{new}})$: 新策略的真实期望累积折扣回报;
- $L_{\pi_{\text{old}}}(\pi_{\text{new}})$: 固定旧策略状态分布下的代理目标 (重要性采样加权 advantage);

- $\frac{4\epsilon\gamma}{(1-\gamma)^2}\alpha^2$: 新旧策略 TV 距离的惩罚项，量化用旧分布近似新分布的误差；
- $\epsilon = \max_{s,a} |A_\pi(s,a)|$: advantage 函数的绝对值上界；
- $\alpha = D_{\text{TV}}^{\max}(\pi_{\text{old}}, \pi_{\text{new}})$: 新旧策略最大 TV 距离。

只要代理目标提升量超过惩罚项，即 $L_{\pi_{\text{old}}}(\pi_{\text{new}}) - \frac{4\epsilon\gamma}{(1-\gamma)^2}\alpha^2 \geq \eta(\pi_{\text{old}})$ ，下界上升，策略性能单调改进。TRPO 的设计思路是：最大化 L （鼓励 advantage 为正的动作为正的动作）同时约束策略变化幅度，即在“可信域”内寻找最优更新：

$$\max_{\pi_{\text{new}}} L_{\pi_{\text{old}}}(\pi_{\text{new}}) \quad \text{s.t.} \quad D_{\text{KL}}^{\max}(\pi_{\text{old}}, \pi_{\text{new}}) \leq \delta.$$

需要注意的是，这个定理更像是 TRPO 的设计坐标系，而不是对实际训练效果的完整解释。如果只停留在“存在足够小的更新步长可以带来局部改进”这一层，普通 policy gradient 在精确梯度和足够小步长下也有类似的局部优化直觉。真正困难的是：深度强化学习中使用的是有限样本估计、非线性神经网络策略和高方差 advantage，步长过小会导致学习极慢，步长稍大又可能让采样分布剧烈漂移。因此 TRPO 的关键不是单调改进公式本身，而是把“不要离旧策略太远”变成一套可执行的更新规则：用 KL 描述策略分布距离，用 Fisher 信息矩阵修正梯度方向，再用共轭梯度和线搜索把这个规则落实到大规模神经网络上。

2.2.2 向任意随机策略的推广

问：与 Theorem 4.1 对比，TRPO 如何推广到所有随机策略类？

答：Kakade–Langford 的 Theorem 4.1 要求新策略为混合形式 $\pi_{\text{new}} = (1 - \alpha)\pi_{\text{old}} + \alpha\pi'$ ，偏离旧策略的概率显式等于 α ，分析局限于此混合策略类，不适用于神经网络参数化的一般策略。TRPO 令

$$\alpha = \max_s D_{\text{TV}}(\pi_{\text{old}}(\cdot|s), \pi_{\text{new}}(\cdot|s))$$

作为“单步采到不同动作的概率上界”，不要求 π_{new} 有特定结构，并重新分析了新旧策略下状态访问分布的偏移 $\|d^{\pi_{\text{new}}}(s) - d^{\pi_{\text{old}}}(s)\|$ ，证明时间维度上的累积误差可由 $O(\alpha^2)$ 控制，从而将性能界推广到任意随机策略（包括神经网络参数化的高斯策略）。

2.2.3 Surrogate Objective 的实际设计

问：surrogate objective $L_{\pi_{\text{old}}}(\pi_{\text{new}})$ 在实际算法中是如何设计的？

答：理论上，代理目标的精确定义为：

$$L_{\pi_{\text{old}}}(\pi_{\text{new}}) = \eta(\pi_{\text{old}}) + \sum_s d^{\pi_{\text{old}}}(s) \sum_a \pi_{\text{new}}(a|s) A_{\pi_{\text{old}}}(s, a).$$

注意到在 $\theta = \theta_{\text{old}}$ 处， L 与真实目标 $\eta(\pi_\theta)$ 有相同的梯度，即 $\nabla_\theta L|_{\theta_{\text{old}}} = \nabla_\theta \eta|_{\theta_{\text{old}}}$ 。因此对 L 做一阶泰勒展开：

$$L(\theta) \approx \eta(\theta_{\text{old}}) + g^\top(\theta - \theta_{\text{old}}), \quad g = \nabla_\theta L|_{\theta=\theta_{\text{old}}},$$

常数项 $\eta(\theta_{\text{old}})$ 不影响优化方向，可直接对梯度项 $g^\top(\theta - \theta_{\text{old}})$ 最大化。

由于真实状态访问分布 $d^{\pi_{\text{old}}}(s)$ 无法精确获得，工程实现中使用旧策略采样数据做 Monte Carlo 估计，并通过重要性采样修正采样偏差：

$$\hat{L}(\theta) = \frac{1}{N} \sum_{t=1}^N \underbrace{\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}}_{r_t(\theta)} \hat{A}_t,$$

其中 $\hat{A}_t$ 由 GAE 估计。代码实现：先保存采样时的 `old_log_probs`，更新时令 `ratio = torch.exp(new_log_probs - old_log_probs)`，最后 `surrogate = (ratio * advantages).mean()`。

2.2.4 KL 约束与线搜索的必要性

问：为什么 TRPO 需要约束 KL divergence 而不是定理中的 TV 距离？为什么需要 line search？

答：

为什么用 KL 代替 TV。 由 Pinsker 不等式：

$$D_{\text{TV}}(p, q) \leq \sqrt{\frac{1}{2} D_{\text{KL}}(p||q)},$$

约束 KL 散度即可隐式约束 TV 距离，且约束更紧。此外，KL 散度对高斯策略等常见分布具有闭式表达且处处可微，而 TV 距离在连续空间中不光滑，难以高效求梯度或做二阶近似。

Fisher 信息矩阵与自然梯度。 在当前参数 θ_{old} 邻域内，对 KL 散度做二阶泰勒展开：

$$D_{\text{KL}}(\theta_{\text{old}}, \theta) \approx \frac{1}{2} \Delta\theta^\top F \Delta\theta, \quad F = \nabla_\theta^2 D_{\text{KL}}(\theta_{\text{old}}, \theta) \Big|_{\theta=\theta_{\text{old}}} = \mathbb{E} \left[\nabla_\theta \log \pi_\theta \nabla_\theta \log \pi_\theta^\top \right] \Big|_{\theta_{\text{old}}}$$

其中 F 为 Fisher 信息矩阵 (FIM)。信赖域约束化为椭球约束 $\frac{1}{2} \Delta\theta^\top F \Delta\theta \leq \delta$ 。在局部线性化目标 $\max_{\Delta\theta} g^\top \Delta\theta$ 下，最优方向为自然梯度方向 $\Delta\theta_{\text{base}} = F^{-1}g$ 。这说明 TRPO 与自然策略梯度 (Natural Policy Gradient, NPG) 关系很近：二者都不是在参数空间的欧氏距离下直接走普通梯度，而是在策略分布的 KL 几何下修正下降方向 [4, 3]。从 mirror descent 角度看，KL 可以被理解为一种 Bregman divergence，策略更新是在“线性化目标 + 邻近项”之间折中；后续 MDPO 工作也明确将 TRPO、PPO 等 trust-region 方法放到 mirror descent 框架下比较 [5]。

满足约束的最大步长为：

$$\Delta\theta^* = \sqrt{\frac{2\delta}{\Delta\theta_{\text{base}}^\top F \Delta\theta_{\text{base}}}} \Delta\theta_{\text{base}}.$$

但神经网络策略的参数数量很大，显式构造并求逆 F 的代价不可接受，正如牛顿法在大规模模型中难以直接求逆 Hessian 一样。因此代码中只实现 Hessian-vector product: `kl_product(v)` 计算 Fv ，再用共轭梯度近似求解线性方程 $Fx = g$ 。这样无需保存完整矩阵，就能得到 $x \approx F^{-1}g$ 。本实验实现中的阻尼项 `return product + vector * 0.1` 还相当于对 F 加上 λI ，用于改善数值条件。

为什么还需要 line search。 二阶近似仅在 θ_{old} 附近局部有效，非线性网络和有限 batch 估计误差会导致实际 KL 超过 δ 或 surrogate 不提升。因此采用回溯线搜索：

$$\theta_k = \theta_{\text{old}} + \beta^k \Delta\theta^*, \quad \beta \in (0, 1),$$

逐步缩小步长，直到同时满足：(1) $D_{\text{KL}}^{\text{real}}(\theta_{\text{old}}, \theta_k) \leq \delta$ ；(2) surrogate objective 不下降。两个条件缺一不可——仅满足 KL 约束可能对应 surrogate 下降；仅满足 surrogate 提升可能违反信赖域边界。若所有候选步长均失败，则回滚参数。相对自然梯度方向和共轭梯度求解，line search 更像是对二阶近似的工程兜底；它很重要，但并不是 TRPO 区别于普通 policy gradient 的唯一核心。

3 探索部分

3.1 不同环境的训练难度

本文选择 **Humanoid-v5**、**HalfCheetah-v5** 和 **Walker2d-v5** 进行测试。

三者训练难度并不一致。在完整 rollout 实验末期，三者平均回报分别约为 1083、4276 和 4718。**HalfCheetah** 与 **Walker2d** 的 reward 曲线持续上升并收敛至较高水平；**Humanoid** 的回报提升较慢且最终值显著较低，说明其高维状态（348 维）、动作空间及复杂身体协调要求显著提高了策略优化难度，同样超参数下 TRPO 很难在相同训练步数内达到同等水平。

3.2 Trajectory 与 Rollout 的样本利用率差异

按实际有效 transition 占采集 transition 的比例计算，**HalfCheetah-v5** 中两种模式的差异最明显：rollout 利用率为 100%，trajectory 平均利用率仅为 62.50%，相差 37.50 个百分点。

原因在于 **HalfCheetah** 的 episode 通常运行至 1000 步上限，多个并行环境的结束时间高度同步；**TrajectoryRollout** 只需存满 10 条轨迹，当第二批 8 个环境同时完成时，仅有 2 条能够写入缓冲区，其余已采集轨迹不参与本次策略更新。

3.3 Advantage 与 Return 的计算差异

两种模式均采用 GAE (Generalized Advantage Estimation)：

$$\delta_t = r_t + \gamma(1 - d_t)V(s_{t+1}) - V(s_t), \quad A_t = \delta_t + \gamma\lambda(1 - d_t)A_{t+1},$$

并令 $R_t = A_t + V(s_t)$ 。主要区别在于：

- **Trajectory 模式**：分别对每条完整轨迹做反向递推，并使用 masks 去除 padding，episode 边界处理更精确。
- **Rollout 模式**：对每个并行环境的固定长度片段做反向递推，所有 transition 均参与更新，无需 mask，但末端若未自然终止则忽略末端 bootstrap，引入 truncation bias。

当前实现中，两种模式均将采样片段最后一步的下一状态价值设为 0；对自然终止的 trajectory 这是合理的，而对未终止即被截断的 rollout 会产生 advantage 与 return 的估计偏差，这也部分解释了为什么在高维任务中 trajectory 有时表现更好。

3.4 Advantage Normalization 的影响

3.4.1 两组实验平均回报曲线差异

本节固定使用 rollout 模式，仅改变 `advan_norm`。如图 1 所示，不同环境的结果差异很大：

- HalfCheetah：不归一化组最终测试平均回报约为 4608，高于归一化组的 4276；
- Humanoid：两组最终回报几乎相同，分别约为 1085 和 1083；
- Walker2d：归一化组最终达到约 4718，而不归一化组长期停留在约 355，几乎未能学会有效行走策略。

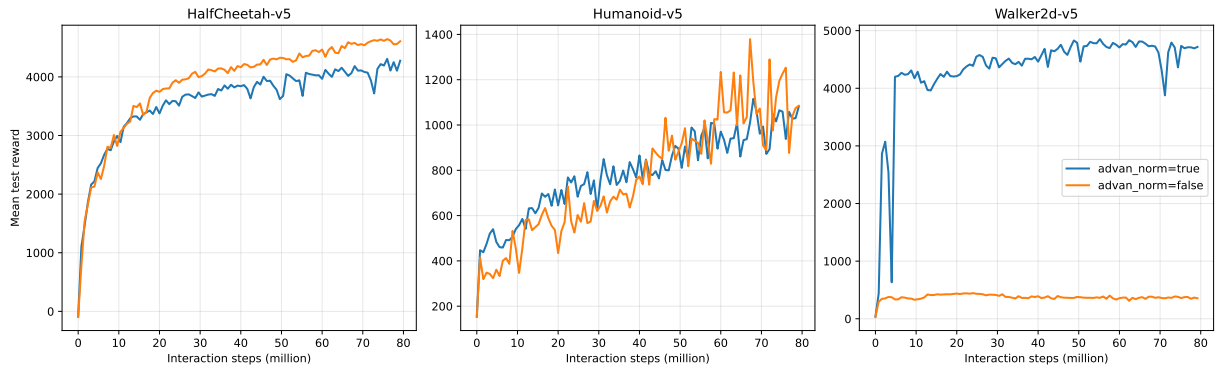


Figure 1: 三个环境中 advantage normalization 对测试平均回报曲线的影响

3.4.2 Advantage Normalization 对训练稳定性的影响

总体而言，advantage normalization 提高了跨环境训练的鲁棒性，但并非在每个环境中都让 reward 曲线更平滑。最明显的例子是 Walker2d：不归一化时 surrogate objective 只有 86.6% 的记录为有限值，其有限值的最大绝对值达约 9.83×10^{33} ；同时平均 KL 仅为 0.00213，约 4.0% 的更新中 KL 接近 0，说明大量策略更新退化或失败。归一化后 surrogate objective 全部为有限值，平均 KL 为 0.00881，能够较稳定地利用 $\delta = 0.01$ 的信赖域。Humanoid 中，归一化组最后十次测试回报的标准差约为 57.1，明显低于不归一化组的 123.8。

图 2 展示了三个环境的 KL divergence 曲线（上）与 surrogate objective 绝对值（下，对数坐标）。需注意 Walker2d 不归一化时 surrogate 量级异常，因此更应关注是否出现非有限值以及 KL 是否退化，而不能只比较绝对大小。

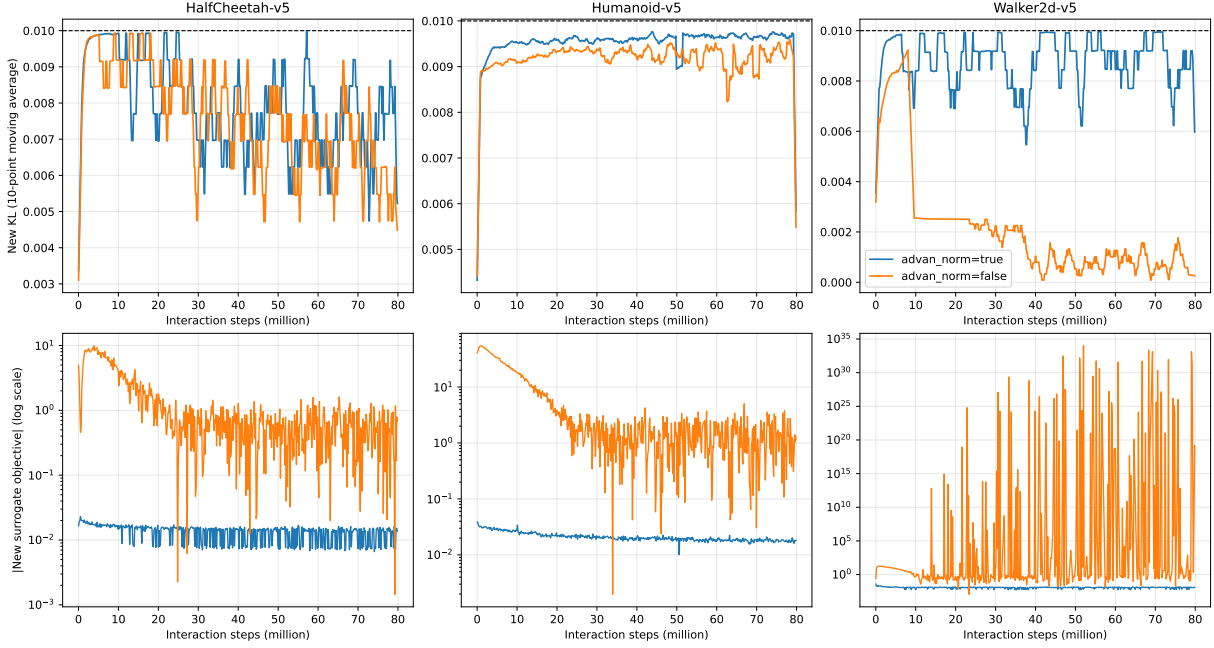


Figure 2: Advantage normalization 对 KL divergence 与 surrogate objective 的影响

3.4.3 Advantage Normalization 是否总是带来性能提升

不是。它在 Walker2d 中带来了决定性提升，但在 Humanoid 中最终性能基本不变，在 HalfCheetah 中不归一化组反而取得了更高的最终回报。

可能原因：TRPO 已通过 KL 约束和 line search 限制策略更新幅度；当原始 advantage 的尺度本身较稳定时，额外归一化并非必需。减去 batch 均值还会改变有限样本下的梯度方向，并引入 batch 统计噪声。因此归一化更像是一种提高数值鲁棒性的机制，而不是必然提升最终性能的方法。

3.5 Trajectory 与 Rollout 的样本利用率比较

3.5.1 每次策略更新使用的 Transition 数量

rollout 模式配置了 8 个并行环境和 `interact_per_epoch=2000`，因此每次更新固定采集并使用 $8 \times 2000 = 16000$ 个 transition。trajectory 模式每次收集 10 条轨迹，有效 transition 数等于 `masks.sum()`，会随 episode 长度变化。对全部 5000 次策略更新取平均后，HalfCheetah、Humanoid 和 Walker2d 每次分别实际使用约 10000、884.81 和 6224.58 个有效 transition；对应的平均采集数分别为 16000、1284.45 和 8780.30。

3.5.2 哪种模式样本利用率更高

如图 3 所示，rollout 中所有固定长度片段都会参与更新，三个环境利用率均为 100%。trajectory 只使用成功写入 `TrajectoryRollout` 的完整轨迹，三个环境平均利用率分别为 62.50%、69.04% 和 70.75%。因此按”参与更新的有效 transition 数 / 实际采集 transition 数”定义，rollout 的样本利用率更高。

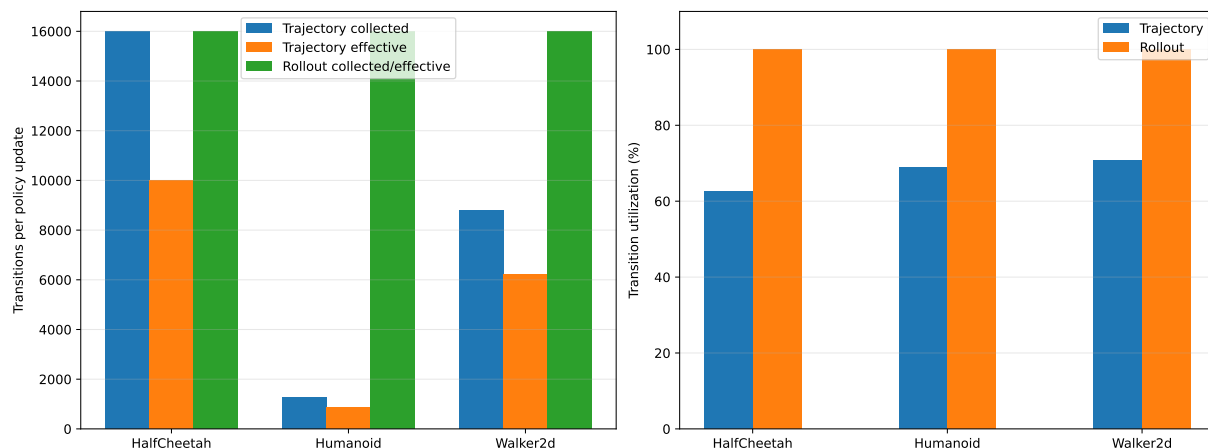


Figure 3: Trajectory 与 rollout 每次更新的 transition 数量及样本利用率

3.5.3 样本利用率更高是否意味着训练效果更好

不一定。如图 4 所示，在相同交互步数处，HalfCheetah 的 trajectory 与 rollout 回报分别约为 1727 和 4276，高利用率 rollout 明显更好；但 Humanoid 在约 6.34×10^6 个交互步时，利用率较低的 trajectory 回报约为 655，高于 rollout 的约 459；Walker2d 在约 4.33×10^7 步时，trajectory 与 rollout 分别约为 4713 和 4657。

这说明样本数量和样本质量需要同时考虑：rollout 不丢弃 transition，但固定长度截断可能引入 GAE 边界偏差；trajectory 会浪费并行采集产生的部分数据，却保留了更完整的 episode 边界。因此，更高的 transition 利用率并不保证更高的 reward。

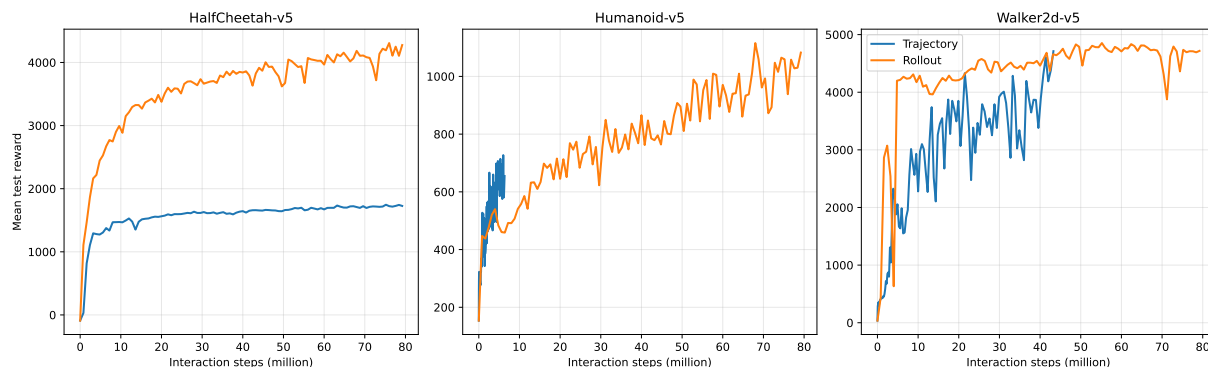


Figure 4: Trajectory 与 rollout 的测试平均回报曲线

4 总结与反思

collect_trajectories 的实现是否会改变 TRPO 的 on-policy 特性？

不会。trajectory 与 rollout 模式均使用当前策略采集数据，并在一次策略更新后清空对应缓冲区，不会重复使用旧策略生成的样本。因此，collect_trajectories 仅改变采样停止条件和数据组织形式，不改变 TRPO 的 on-policy 特性。

如果只能保留一种采样方式，你会选择 trajectory 还是 rollout？

如果只能保留一种，我会选择 rollout。最直接的原因是它采集的 transition 都会参与本次更新，三个环境中的样本利用率都是 100%，而 trajectory 只有约 62.50% ~ 70.75%。从实验结果看，rollout 在 HalfCheetah 上明显优于 trajectory，在 Walker2d 上最终性能也很好，说明固定长度截断带来的 advantage 估计偏差不一定会成为主要问题。另外，rollout 每次更新固定使用 16000 个 transition，训练批量更容易控制，比较不同实验时也更方便。

但这个选择并不是说 trajectory 没有价值。Humanoid 和 Walker2d 在部分相同交互步数下，trajectory 的回报反而更高，说明完整轨迹确实可能带来更准确的 advantage。只是综合样本利用率、实现复杂度和三个环境的整体结果，rollout 更适合作为默认采样方式。如果继续改进，可以给 rollout 末端加入正确的 value bootstrap，尽量减小固定长度截断造成的偏差。

完成实验报告的撰写后，你对 TRPO 算法是否有了新的感悟或认识？

我一开始对 TRPO 的理解主要停留在“用 KL 散度限制策略更新”这一句话上，后来重新读论文和代码后，感觉这句话仍然正确，但并不完整。TRPO 论文中的单调改进推导提供了一个很漂亮的理论外壳：真实回报难以直接优化，于是构造 surrogate objective，并用策略距离惩罚状态分布偏移。但这些推导依赖精确期望、精确 advantage 和理想化的约束求解；一旦进入神经网络和有限样本训练，理论本身并不能直接回答“为什么 vanilla policy gradient 一定不行、TRPO 一定行”。如果只追求局部改进，普通梯度法配合足够小的步长也有局部上升的直觉，问题在于这个步长往往小到训练效率很差，而且高方差采样会让“足够小”很难判断。

因此我现在更倾向于把 TRPO 理解为一种可实现的 natural policy gradient / mirror descent 方法，而不是只把它理解成单调改进定理的直接应用。普通梯度在参数空间的欧氏几何下更新，而 TRPO 用 KL 所诱导的 Fisher 信息矩阵重新度量“一步走多远”；这和牛顿法用 Hessian 调整方向有相似之处，只是这里的距离来自策略分布而不是参数向量本身。真正让它能用于深度策略网络的，是一系列实现选择：不显式构造 Fisher 矩

阵，只通过 Hessian-vector product 计算 Fv ；用共轭梯度近似求解 $Fx = g$ ；再用有限次数 line search 检查实际 KL 和 surrogate。从这个角度看，TRPO 的贡献不只是一个不等式，而是把一个几何上合理但计算上昂贵的更新变成了可运行的算法。

这次实验也让我更重视 implementation choice。Walker2d 关闭 advantage normalization 后，surrogate objective 出现极端量级和非有限值，说明一个看似“只是标准化”的细节会直接改变训练是否可用。trajectory 模式中的 masks、rollout 末端是否 bootstrap、buffer 满时 pos 如何解释，也都不是论文主公式里最显眼的內容，但它们会决定最后算出来的 advantage、KL 和梯度是否可信。后续许多深度强化学习工作也反复说明，代码级优化和实现细节会显著影响算法表现 [6]。所以相比把复杂数学推导当作算法优越性的全部原因，我认为更实际的学习方式是：先理解理论给出的设计原则，再检查这些原则在代码中如何被近似、被简化、被数值技巧支撑。

针对本次算法实现，你是否有什么想法和想改进的建议？

这次实现中，我卡得最久的地方不是某个公式本身，而是公式和代码中的数据形状、缓冲区状态对应不上。例如 rollout 缓冲区刚好填满时，`ReplayBuffer.pos` 会回到 0，如果继续把 pos 当作有效长度，就会取到空数据，最后在计算 log probability 或 reshape 时才报错。这个错误离真正原因比较远，排查起来很耗时间。后来需要根据 full 判断有效长度，缓冲区满时使用 `buffer_size`。另外，服务器连接中断、SwanLab 无法连接以及长时间实验失败后重新运行，也占用了不少时间。

README 目前主要介绍了环境安装和两个运行脚本，但对配置项、输出目录和常见错误解释得比较少。我建议补充一组能够快速验证代码的短实验命令，并说明 `collect_traj`、`buffer_name`、`interact_per_epoch` 和 `advan_norm` 等参数的作用；同时给出服务器无法连接 SwanLab 时使用 TensorBoard 保存日志、再复制到本地分析的备用流程。代码中的 TODO 即使已经实现仍然保留，也容易让人误以为对应部分还没有完成，最好在实现后删除或改成普通注释。

我认为这次任务的难点属于“看懂公式不算特别难，但正确实现并验证比较难”。GAE、共轭梯度和 line search 单独看都能理解，真正困难的是保证 mask、episode 边界、buffer 长度和 tensor shape 全部正确，并通过多个环境的长时间实验确认实现没有隐藏问题。如果项目能够提供少量针对 advantage 计算、满缓冲区和 trajectory mask 的单元测试，会减少很多排查时间。

5 总体结论

本实验完成了 TRPO 算法在三个 MuJoCo v5 环境中的系统性验证，主要结论如下：

- (1) 三个环境训练难度差异显著: HalfCheetah 最易收敛 (最终回报约 4276), Walker2d 最终性能最高 (约 4718), Humanoid 因高维状态最难训练 (约 1083)。
- (2) Rollout 样本利用率 (100%) 显著高于 trajectory (62% ~ 71%); 但利用率更高不一定带来更好的训练效果——Humanoid 和 Walker2d 中 trajectory 在相同交互步数下表现更好。
- (3) Advantage normalization 对数值稳定性至关重要: Walker2d 关闭归一化后 surrogate objective 出现量级达 10^{33} 的非有限值, 训练几乎失败; 但在 HalfCheetah 中, 不归一化组最终回报反而更高, 说明其效果具有环境依赖性。
- (4) TRPO 的 KL 约束配合共轭梯度和 line search 在实验中整体生效, 但理论上的单调改进在有限样本和二阶近似下并不总能严格保证, advantage normalization 和 rollout 末端 bootstrap 的正确处理同样关键。
- (5) 从实现角度看, TRPO 更像是将 natural policy gradient / mirror descent 思想工程化: 用 Hessian-vector product 和共轭梯度避免显式求逆 Fisher 矩阵, 再通过若干数值技巧保证更新可运行。因此报告中的理论推导应和具体实现选择一起理解。

References

- [1] Schulman, J., Levine, S., Abbeel, P., Jordan, M., & Moritz, P. (2015). Trust Region Policy Optimization. In *Proceedings of the 32nd International Conference on Machine Learning (ICML 2015)*, pp. 1889–1897. PMLR.
- [2] Kakade, S., & Langford, J. (2002). Approximately Optimal Approximate Reinforcement Learning. In *Proceedings of the 19th International Conference on Machine Learning (ICML 2002)*, pp. 267–274.
- [3] OpenAI. Spinning Up: TRPO. <https://spinningup.openai.com/en/latest/algorithms/trpo.html>
- [4] Kakade, S. M. (2001). A Natural Policy Gradient. In *Advances in Neural Information Processing Systems 14*.
- [5] Tomar, M., Shani, L., Efroni, Y., & Ghavamzadeh, M. (2020). Mirror Descent Policy Optimization. *arXiv preprint arXiv:2005.09814*.
- [6] Engstrom, L., Ilyas, A., Santurkar, S., Tsipras, D., Janoos, F., Rudolph, L., & Madry, A. (2020). Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO. *arXiv preprint arXiv:2005.12729*.